NBA COURTVISION

Design, Implementation, and Evaluation of an End-to-End NBA Win-Probability Analytics System

A top-down technical research paper and relearning guide covering system design, data engineering, machine learning, APIs, interface design, containers, cloud deployment, evaluation, limitations, and operations.

PROJECT OWNER / CHANAKYA G
SYSTEM SNAPSHOT / 27 AUGUST 2026
DEPLOYMENT / AWS EC2, US-EAST-1
PUBLIC INTERFACE / NBA COURTVISION

PURPOSE

This document is written to make the complete project reconstructible from first principles. It explains not only what each component does, but why it exists, how information crosses boundaries, what the reported metrics mean, and where the current design should evolve.

Abstract

NBA CourtVision is an end-to-end sports analytics system that converts public ESPN game feeds into play-level home-team win probabilities and an interactive season exploration experience. Its full engineering path includes a retryable Prefect ingestion flow, normalized PostgreSQL warehouse, reproducible feature transformation, game-grouped XGBoost training, persisted evaluation artifacts, batch prediction storage, FastAPI inference, Streamlit interfaces, Docker packaging, and AWS EC2 delivery. A second, self-contained serving path bundles an exported season snapshot with the trained model so the public site does not depend on a continuously running database.

The observed public snapshot contains 1,401 scheduled games, 679,005 play records, and 601,601 scored play states (88.6% coverage) from 2 October 2025 through 14 June 2026. The saved training artifact reports 893,785 training rows and 223,158 test rows drawn from 2,104 and 527 games, respectively. On its held-out game split, the classifier achieved ROC AUC 0.8384, log loss 0.4891, Brier score 0.1642, and a project-defined calibration accuracy of 0.9805. These results demonstrate useful discrimination and generally close calibration within the evaluated snapshot, but they do not establish future-season performance because the split is random at the game level rather than chronological.

The central design lesson is separation of concerns: raw acquisition, durable state, deterministic features, model artifacts, inference contracts, presentation, and deployment each have an explicit boundary. The central scientific lesson is equally important: a probability system must be evaluated as a probability system, with leakage-resistant splitting, proper scoring rules, calibration analysis, and careful claims.

Core thesis
CourtVision is best understood as two connected products: a model-building data platform and a lightweight public analytical application. The saved artifacts are the contract between them.

CONTENTS

Abstract 2

1. Research Question and Product Intent 6

1.1 The analytical question . . . 6
1.2 Product objectives . . . 6
1.3 Non-goals . . . 6
1.4 Success criteria . . . 6

2. System Architecture: Top-Down View 7

2.1 End-to-end information flow . . . 7
2.2 Control plane versus data plane . . . 7
2.3 Architectural trade-off . . . 8

3. Data Acquisition and ETL 8

3.1 Source endpoints . . . 8
3.2 Prefect flow anatomy . . . 8
3.3 Date and event semantics . . . 8
3.4 Normalization rules . . . 8
3.5 Clock transformation . . . 8
3.6 Idempotency and failure behavior . . . 9

4. Warehouse and Data Model 10

4.1 Table responsibilities . . . 10
4.2 Keys and referential integrity . . . 10
4.3 Indexes . . . 10
4.4 Schema ownership nuance . . . 10
4.5 Local persistence caveat . . . 11

5. Dataset Profile 11

5.1 Why 88.6% of plays are scored . . . 11
5.2 Training corpus versus public corpus . . . 11
5.3 Data quality questions to ask . . . 11

6. Feature Engineering and Target Construction 12

6.1 Target label . . . 12
6.2 The twelve-feature contract . . . 12
6.3 Consistency across pathways . . . 12
6.4 Feature redundancy . . . 12
6.5 Late-game ratio behavior . . . 12

7. Training Design and XGBoost 13

7.1 Leakage-resistant split . . . 13
7.2 What remains untested . . . 13

7.3 Model configuration . . . 13
7.4 Gradient boosting mental model . . . 13
7.5 Artifact production . . . 13
8. Evaluation: What the Numbers Mean 14
8.1 ROC AUC . . . 14
8.2 Log loss . . . 14
8.3 Brier score . . . 14
8.4 Project-defined calibration accuracy . . . 14
9. Model Interpretation 15
9.1 Dominant signals . . . 15
9.2 Why importance can mislead . . . 15
9.3 Sanity checks . . . 16
9.4 Calibration by game state . . . 16
10. Inference and API Design 16
10.1 Four inference paths . . . 16
10.2 Batch scoring . . . 16
10.3 FastAPI contract . . . 16
10.4 Latency benchmarking . . . 16
10.5 Contract risks . . . 17
11. Frontend and Analytical Experience 17
11.1 Development interfaces . . . 17
11.2 Public CourtVision interface . . . 17
11.3 Interaction design principles . . . 17
11.4 Public runtime computation . . . 17
12. Containers and Runtime Topology 18
12.1 Full local Docker Compose . . . 18
12.2 Public image . . . 18
12.3 Health and restart semantics . . . 18
12.4 Environment and secrets . . . 18
13. AWS Deployment and Operations 19
13.1 Provisioning sequence . . . 19
13.2 IAM boundary . . . 19
13.3 Current deployment snapshot . . . 19
13.4 Production hardening path . . . 19
13.5 Cost model . . . 19
14. Reliability, Security, and Failure Analysis 20
14.1 Failure-mode table . . . 20
14.2 Existing strengths . . . 20
14.3 Test posture . . . 20

14.4 Minimum future test pyramid . . . 20

15. Reproduction Guide **21**

15.1 Local environment . . . 21
15.2 Ingest a date range . . . 21
15.3 Train and inspect artifacts . . . 21
15.4 Score warehouse plays . . . 21
15.5 Run inference surfaces . . . 21
15.6 Run the public app locally . . . 21
15.7 Deploy with the dedicated AWS profile . . . 21
15.8 Verification checklist . . . 21

16. Design Critique and Roadmap **22**

16.1 Highest-priority correctness improvements . . . 22
16.2 A stronger modeling experiment . . . 22
16.3 A stronger production architecture . . . 22
16.4 What not to add first . . . 22

17. What This Project Teaches **23**

17.1 System-design lessons . . . 23
17.2 Machine-learning lessons . . . 23
17.3 Product lessons . . . 23
17.4 The one-minute explanation . . . 23

Appendix A. Repository Map **24**

Appendix B. Glossary **25**

Study sequence . . . 25

References **26**

1. Research Question and Product Intent

1.1 The analytical question

At any play state in an NBA game, what is the probability that the home team will ultimately win? The system represents a play state with the period, game time remaining, scores, derived margin and pace signals, and whether the current record is a scoring play. It maps that state to a number in [0, 1]. A value of 0.72 means that comparable states, under the learned mapping, favor a home win at roughly 72%. It is not a point-spread prediction, an expected final score, or a guarantee.

1.2 Product objectives

- Acquire repeatable, structured play-by-play data from ESPN endpoints.
- Preserve both normalized analytical columns and original source JSON.
- Train without placing plays from the same game on both sides of evaluation.
- Expose the model through batch, CLI, HTTP, and embedded UI pathways.
- Let a reader move from season overview to game narrative to individual play.
- Package the portfolio demonstration independently from the warehouse.
- Deploy the public application with repeatable infrastructure automation.

1.3 Non-goals

The system is not a live sportsbook engine, player tracking model, causal model, injury-aware forecast, or production-grade high-availability service. It does not consume lineups, possessions, betting markets, player availability, travel, rest, or tracking coordinates. These omissions are not minor details: they define the boundary of the model's knowledge.

1.4 Success criteria

Dimension	Practical criterion	Current evidence
Data	Idempotent historical ingestion	Primary-key upserts; run ledger
Science	Leakage-aware evaluation	Game-level stratified split
Serving	Consistent feature contract	Shared 12-column ordering
UX	Task-oriented exploration	Five-page CourtVision navigation
Operations	Reproducible public launch	Docker + scripted EC2 deployment
Trust	Calibrated probabilistic output	Brier, log loss, 10-bin calibration

2. System Architecture: Top-Down View

The architecture is deliberately hybrid. The warehouse plane supports collection, retraining, batch scoring, and operational introspection. The public plane serves a frozen analytical snapshot plus the model artifact. This reduces cloud dependencies and cost while preserving the full project as a reproducible data system.

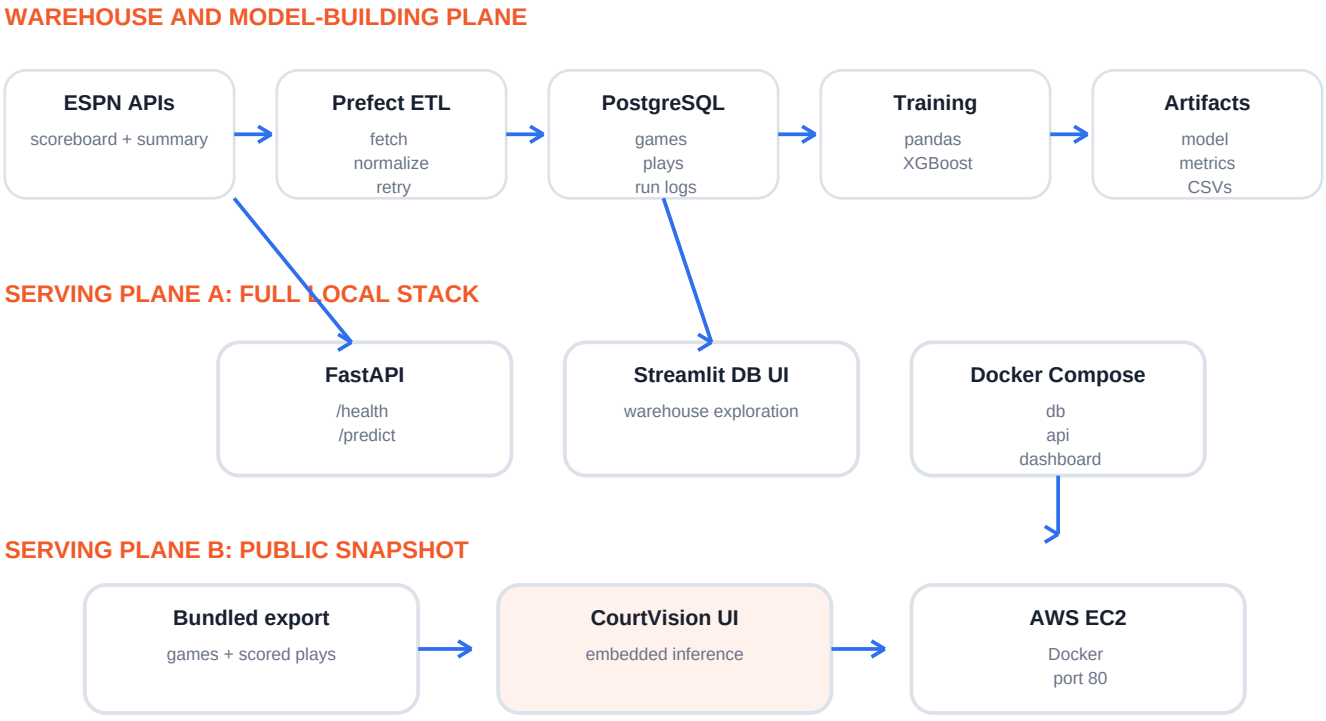

Figure 1. Two-plane architecture. The artifact boundary separates model construction from public serving.

2.1 End-to-end information flow

Stage	Input	Transformation	Durable output
Discover	Date or event IDs	ESPN scoreboard lookup	Unique event IDs
Extract	Event ID	ESPN summary request	Nested JSON
Normalize	Nested JSON	Game/play flattening, clock and margin derivation	Python dictionaries
Load	Normalized records	Transactional upsert	PostgreSQL warehouse
Train	Eligible plays	Feature engineering + game split + XGBoost	Model JSON + evaluation CSV/JSON
Score	Warehouse plays	Vectorized probability inference	model_predictions rows
Export	Games + joined predictions	CSV snapshot	Public data bundle
Serve	Bundle + model	Cached analytics and on-demand inference	Streamlit application
Deploy	Source bundle	Docker build on EC2	Public HTTP endpoint

2.2 Control plane versus data plane

The data plane carries games, plays, features, probabilities, and artifacts. The control plane decides when and how work happens: Prefect retries, CLI arguments, database transactions, Docker health checks, AWS IAM, security groups, instance state checks, and deployment verification. Many early data projects model only the data plane. CourtVision includes enough control-plane machinery to demonstrate how an analytical script becomes an operable system.

2.3 Architectural trade-off

Why bundle the public snapshot?
A live database-backed website would be fresher but requires persistent database operations, migrations, credentials, and additional failure modes. The bundled snapshot is cheaper, simpler, deterministic, and suitable for a portfolio. Its cost is staleness: data changes require re-export and redeployment.

3. Data Acquisition and ETL

3.1 Source endpoints

The ETL uses two ESPN site endpoints under site.api.espn.com: a date-filtered scoreboard endpoint to discover event IDs and a summary endpoint to obtain event metadata and play-by-play records. These are treated as external, failure-prone dependencies. Both fetch tasks use a 30-second request timeout and Prefect retry policy of three attempts with five seconds between attempts.

3.2 Prefect flow anatomy

Function	Responsibility	Design property
ensure_schema	Create tables/indexes; rename incompatible legacy play table	Backward-tolerant initialization
fetch_scoreboard_event_ids	Resolve a YYYYMMDD date to events	Retryable extraction
fetch_event_summary	Download one event payload	Retryable extraction
normalize_game	Flatten header and home/away identities	Schema boundary
normalize_play	Flatten clock, score, type, text, raw JSON	Deterministic transformation
upsert_event	Transactionally upsert game and every play	Idempotent loading
create/finish_ingest_run	Record counts, status, and errors	Operational observability
ingest_play_by_play	Coordinate dates or IDs and fail coherently	Flow orchestration

3.3 Date and event semantics

The command accepts repeated --event-id values or a start/end date. A date range is inclusive. Event IDs are converted to strings, deduplicated with a set, and sorted. With neither input, the flow ingests the current date. A UUID identifies each run and the source field records either dates or the literal event_ids.

3.4 Normalization rules

Game metadata is extracted from the first competition. Home and away teams are chosen by homeAway, with abbreviation-based fallback names. Each play receives a deterministic sequence number from enumeration. If ESPN omits a play ID, the fallback is game_id-sequence_number. Scores are converted into home score margin when possible. raw_json preserves the original play object in PostgreSQL JSONB, enabling later reprocessing without re-fetching.

3.5 Clock transformation

For regulation period p and displayed clock m:s, the implemented remaining-game-time transformation is:

```
clock_seconds = 60*m + s
seconds_remaining = max(4 - p, 0) * 720 + clock_seconds
```

This yields 2,880 seconds at the opening tip and zero at the end of regulation. The current parser assumes a 12-minute period for all periods. For overtime, NBA periods are five minutes, so the parser can overstate overtime time remaining. The public Prediction Lab improves the input convention for overtime, but the upstream ETL should be corrected so training and serving semantics are identical.

3.6 Idempotency and failure behavior

Games use ON CONFLICT(game_id) DO UPDATE. Plays use ON CONFLICT(game_id, play_id) DO UPDATE. Re-ingesting an event therefore refreshes it rather than duplicating it. Each event load occurs inside an engine.begin transaction. If an exception escapes the event loop, the run ledger is marked failed with partial success counts and the exception is re-raised.

Engineering principle
Idempotency makes retries safe. A retry policy without idempotent writes can multiply data; an upsert without a run ledger can hide partial failure. The project uses both patterns.

4. Warehouse and Data Model

PostgreSQL is the authoritative state store for the full pipeline. The schema separates descriptive entities, granular events, run metadata, model runs, and predictions. This makes it possible to update one concern without rewriting the others.

Figure 2. Logical entity relationships. model_training_runs is artifact metadata rather than a direct game foreign key.

4.1 Table responsibilities

Table	Grain	Primary key	Purpose
games	One scheduled event	game_id	Teams, date, status, names
play_by_play	One event record within a game	game_id + play_id	Clock, score, type, text, raw source
ingest_runs	One ETL attempt	run_id	Dates, counts, status, errors
model_training_runs	One training execution	run_id	Split sizes, metrics, artifact path
model_predictions	One model's score for one play	game_id + play_id + model_path	Version-aware probability history

4.2 Keys and referential integrity

The composite play primary key recognizes that a play ID is meaningful within a game. Predictions reference that composite key and cascade on deletion. The model_path is part of the prediction primary key, allowing multiple saved model versions to score the same play without overwriting each other. The live warehouse UI selects the most recently scored model path per game.

4.3 Indexes

idx_play_by_play_game_sequence supports the dominant access pattern: reconstruct one game's ordered narrative. idx_play_by_play_period supports period filtering. idx_model_predictions_game begins with game_id and orders scored_at descending, supporting recent prediction retrieval. These indexes reflect query shapes rather than indexing every column.

4.4 Schema ownership nuance

The project contains SQLAlchemy ORM declarations for games, play_by_play, and ingest_runs, but the active ETL also owns raw CREATE TABLE statements. Training and scoring create their own tables lazily. This works for a compact project, but a production successor should centralize schema evolution in migrations such as Alembic to avoid drift between ORM declarations and SQL DDL.

4.5 Local persistence caveat

The development Docker Compose file does not define a named PostgreSQL volume. Removing the database container may therefore remove the warehouse. Persistent projects should mount /var/lib/postgresql/data to a named volume and manage backups.

5. Dataset Profile

The following measurements come directly from the bundled public CSV snapshot, not from README claims. The observed window differs slightly from the prose documentation and should be treated as the reproducible ground truth for the current public deployment.

Measure	Observed value	Interpretation
Scheduled games	1,401	1,397 final; 4 postponed
Game date window	2025-10-02 to 2026-06-14 UTC	Exported snapshot boundary
Unique team abbreviations	37	Includes all event abbreviations in export
Play records	679,005	Mean 486.0 per completed game
Scored play states	601,601	88.60% model coverage
Unscored play states	77,404	All lack seconds_remaining
Home win rate	55.48%	Completed games in public snapshot
Average total score	229.68	Final home + away points
Mean absolute margin	13.26 points	Average final separation
Close-game share	24.70%	Final margin <= 5
Overtime play rows	4,231	Periods 5 and 6

5.1 Why 88.6% of plays are scored

The training and scoring queries require non-null home score, away score, score margin, and seconds remaining. In the current export, scores and margins are present on all rows, while exactly 77,404 rows lack seconds_remaining. Those same rows lack a stored probability. This is a clean example of how one upstream field controls downstream model coverage.

5.2 Training corpus versus public corpus

The saved training metrics report 2,631 total games (2,104 train + 527 test) and 1,116,943 eligible rows. The public export contains 1,401 scheduled games and 601,601 scored rows. Therefore the model artifact was trained from a broader warehouse snapshot than the public season bundle. This is acceptable if intentional, but model cards and UI copy should say so explicitly; otherwise a reader may assume the evaluation and public data populations are identical.

5.3 Data quality questions to ask

- Are postponed or cancelled games excluded before final-label construction?
- Does the final observed scored play always contain the true final score?
- Are ESPN corrections re-ingested, and can they change labels?
- Are overtime clocks encoded consistently across training, API, and UI?
- Do event-specific team abbreviations explain the count above the 30 NBA franchises?
- Is the export reproducible from a recorded warehouse version and model hash?

6. Feature Engineering and Target Construction

6.1 Target label

For each game, the training code sorts plays by game_id and sequence_number, takes the last eligible play, and defines home_win = 1 when final_home_score > final_away_score. This single game outcome is joined back to every eligible play in that game. The model thus learns P(home eventually wins | current play state).

6.2 The twelve-feature contract

Feature	Definition	Intuition
period	Current period number	Game phase
seconds_remaining	Regulation-style game seconds remaining	Time pressure
home_score	Current home points	Score state and pace
away_score	Current away points	Score state and pace
score_margin	home_score - away_score	Signed advantage
abs_score_margin	absolute margin	Game closeness
total_score	home_score + away_score	Game pace / progress
score_margin_per_minute	margin / max(minutes left, 1/60)	Lead relative to remaining opportunity
is_home_leading	1 if margin > 0	Explicit lead state
is_tied	1 if margin = 0	Special boundary state
is_late_game	1 if seconds <= 300	Final-five-minute regime
is_scoring_play	1 if source marks scoring play	Immediate event context

6.3 Consistency across pathways

The same column order appears in features.py, train_model.py, score_predictions.py, api.py, and the public application. XGBoost consumes columns positionally and by name, so consistent construction is critical. A stronger production pattern would serialize the transformation and model as one pipeline artifact, or enforce a schema hash at load time.

6.4 Feature redundancy

Several variables are deterministic transformations of the same score state: score_margin, abs_score_margin, is_home_leading, and is_tied. Tree models can use redundant thresholds effectively, but importance becomes harder to interpret because correlated features divide or concentrate credit. Home_score, away_score, and total_score are also algebraically related. Feature importance should therefore be read as model usage, not causal influence.

6.5 Late-game ratio behavior

score_margin_per_minute becomes extremely large as seconds_remaining approaches zero. The implementation clips the denominator to one second, avoiding division by zero but still creating heavy tails. A more stable alternative could use margin divided by sqrt(time + c), bucketed time, or monotonic constraints informed by basketball logic.

7. Training Design and XGBoost

7.1 Leakage-resistant split

A naive row-level random split would leak nearly identical states from the same game into training and test sets. CourtVision first groups by game, stratifies game IDs by home-win label, samples approximately 20% of each class into the test set, and then assigns every play from a game to the same side. This is a strong and necessary design choice.

7.2 What remains untested

The game split is random rather than chronological. It tests generalization to held-out games from the same broad population, not forward generalization to a future season or changed league environment. A production evaluation should use rolling-origin or season-based validation and report performance by month, quarter, score margin, and time remaining.

7.3 Model configuration

Parameter	Value	Effect
objective	binary:logistic	Probability of home win
n_estimators	250	Number of boosted trees
max_depth	4	Moderate interaction complexity
learning_rate	0.05	Conservative contribution per tree
subsample	0.9	Row sampling for regularization
colsample_bytree	0.9	Feature sampling per tree
eval_metric	logloss	Probability-sensitive optimization monitor
random_state	42 by default	Reproducible sampling/model behavior
n_jobs	2	Bounded CPU parallelism

7.4 Gradient boosting mental model

XGBoost builds an additive ensemble of decision trees. The first trees learn broad rules such as whether the home team is leading. Later trees focus on residual errors, learning conditional refinements such as how a three-point lead means something different with 30 seconds remaining than with 30 minutes remaining. For binary logistic output, the ensemble produces a log-odds score F(x), converted to probability by the sigmoid:

```
p(home win | x) = 1 / (1 + exp(-F(x)))
F(x) = sum_k f_k(x), where each f_k is a decision tree
```

7.5 Artifact production

A successful training run writes the XGBoost model JSON, training_metrics.json, calibration_bins.csv, and feature_importance.csv. It also records split sizes, metrics, model path, and the full metrics JSON in model_training_runs. These artifacts make the model inspectable without re-running training.

8. Evaluation: What the Numbers Mean

Metric	Result	Interpretation
ROC AUC	0.8384	Useful ranking discrimination; 0.5 is random
Log loss	0.4891	Penalizes confident wrong probabilities
Brier score	0.1642	Mean squared probability error
Calibration accuracy	0.9805	1 minus weighted 10-bin absolute calibration error
Test size	223,158 rows / 527 games	Grouped held-out evaluation population

8.1 ROC AUC

ROC AUC measures how often a randomly selected positive state receives a higher score than a randomly selected negative state. It ignores probability calibration and the sequential dependence among play rows. Because every game contributes many correlated rows, row-count confidence intervals would be misleading; uncertainty should be bootstrapped by game.

8.2 Log loss

```
LogLoss = -(1/N) * sum_i [ y_i log(p_i) + (1-y_i) log(1-p_i) ]
```

Log loss strongly penalizes confident errors. A 0.99 prediction on a home loss is far more costly than a 0.60 prediction on the same outcome. This is appropriate when probability quality matters.

8.3 Brier score

```
Brier = (1/N) * sum_i (p_i - y_i)^2
```

The Brier score is bounded between 0 and 1 for binary events, with lower values better. It combines calibration and discrimination. It should be compared with simple baselines, such as the unconditional home-win rate and a score/time heuristic, which the current artifact does not report.

8.4 Project-defined calibration accuracy

The code bins predictions into ten equal-width intervals, computes the absolute difference between mean prediction and observed home-win rate in each bin, weights by samples, and reports one minus that error. The weighted error is 0.01946, producing 0.98054. This is understandable but is not a universal metric named calibration accuracy; future reports should call it 1 - weighted expected calibration error and also report ECE directly.

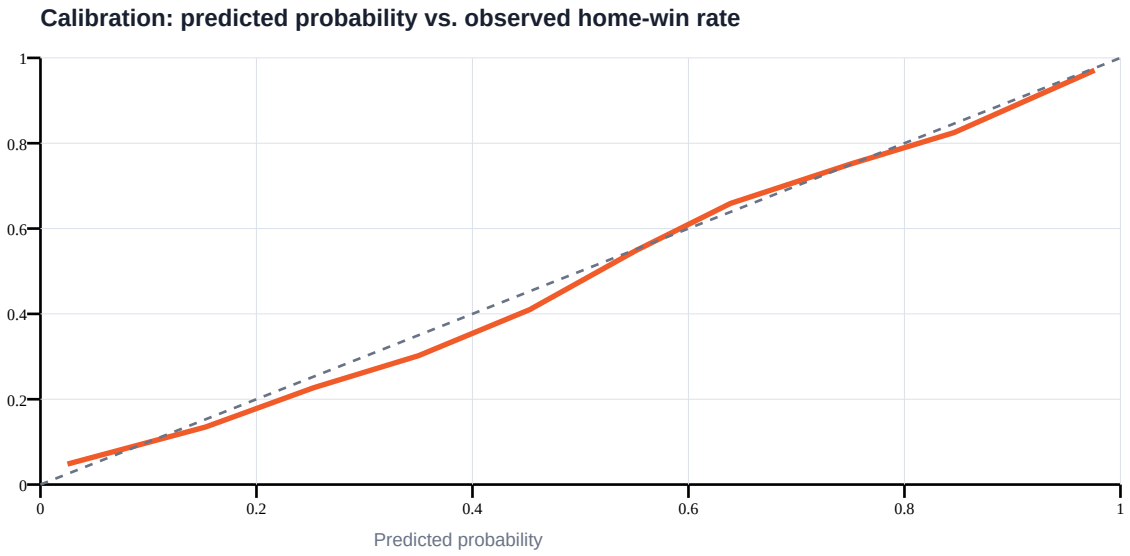

Figure 3. Calibration curve derived from the stored ten-bin artifact. The dashed line is perfect calibration.

9. Model Interpretation

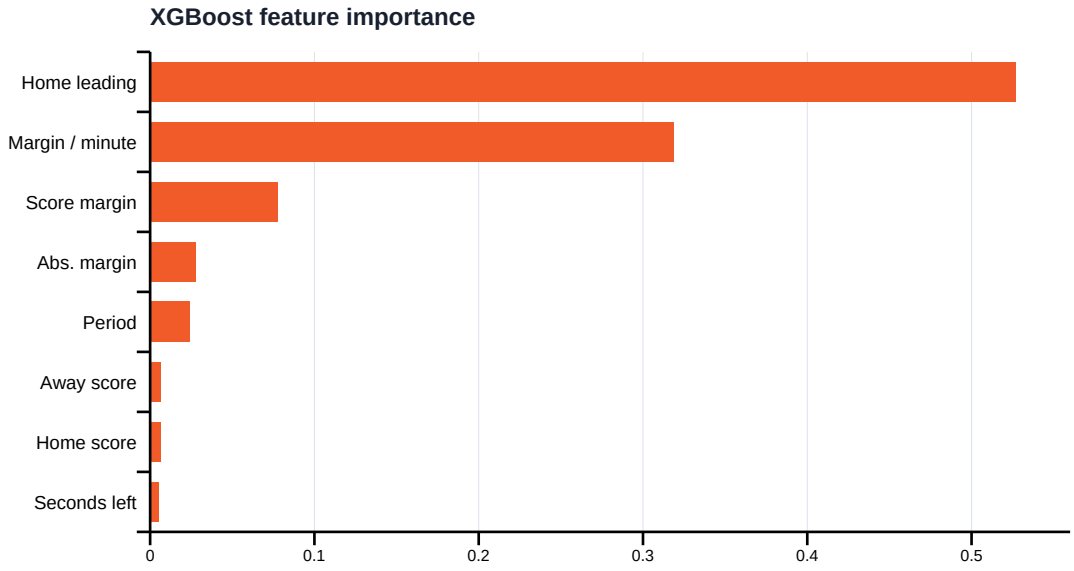

Figure 4. Top eight stored feature importances. Values are model-specific usage importance, not causal effects.

9.1 Dominant signals

is_home_leading accounts for 52.69% of stored importance and score_margin_per_minute for 31.87%. Together they account for approximately 84.56%. This matches basketball intuition: who leads and how large that lead is relative to remaining time dominate outcome probability.

9.2 Why importance can mislead

The feature set contains correlated transformations. A binary home-leading flag may receive a large share because it provides an efficient early tree split, while score_margin adds refinements. This does not mean being home-leading causes victory with the reported strength, nor that low-importance features have no value. Permutation importance measured by game, SHAP analysis, and ablation studies would provide stronger explanations.

9.3 Sanity checks

- At identical time, probability should generally rise with home score margin.
- At identical positive margin, probability should generally rise as time expires.
- A tied opening state should remain near the learned home-court prior.
- Extremely late ties should not become deterministic.
- Overtime inputs should use five-minute period semantics.
- Impossible combinations, such as period 1 with zero total game seconds, should be rejected or normalized.

9.4 Calibration by game state

Global calibration can hide local errors. A more rigorous model card should stratify reliability by quarter, close versus blowout state, overtime, home/away winner, and prediction extremity. The 0.3-0.4 bin currently has the largest stored absolute gap, about 4.8 percentage points.

10. Inference and API Design

10.1 Four inference paths

Path	Entry point	Use case
Local CLI	predict_model.py	One-off reproducible debugging
Batch scoring	score_predictions.py	Persist probabilities for warehouse plays
HTTP API	api.py	External synchronous requests
Embedded public UI	hf_space/app.py	Instant interactive demonstration

10.2 Batch scoring

The scorer selects eligible warehouse rows, derives the same 12 features, loads the model once, computes probabilities vectorially, and upserts in batches of 5,000 by default. The composite key includes model_path, preserving per-model histories. A --game-id option supports targeted rescoring.

10.3 FastAPI contract

PredictionRequest validates period 1-8, seconds_remaining 0-2880, and non-negative scores. Both POST /predict and GET /predict are supported. The response returns probability, measured model inference time, model path, and the computed feature dictionary. /health distinguishes status ok from model_not_loaded. If the artifact is absent, predictions return HTTP 503.

```
POST /predict
{
"period": 4,
"seconds_remaining": 300,
"home_score": 98,
"away_score": 95,
"scoring_play": false
}
```

10.4 Latency benchmarking

benchmark_api.py warms the endpoint, sends sequential POST requests, counts failures, and reports throughput plus mean, p50, p95, p99, and max client-observed latency. This is a useful smoke benchmark, but it does not model concurrency, connection pooling, autoscaling, cold starts, or sustained load. Inference_ms inside the API excludes HTTP and feature-construction overhead, while the benchmark measures full round-trip time.

10.5 Contract risks

The API permits semantically inconsistent inputs, such as period 2 with ten seconds remaining in the whole game. It also accepts overtime periods but retains a 2,880-second maximum without period-aware constraints. A next version should validate clock semantics, publish an OpenAPI example, add a model_version field, and return an explicit prediction timestamp.

11. Frontend and Analytical Experience

11.1 Development interfaces

src/dashboard.py provides warehouse operations: overall game/play/prediction counts, games by date, ingest runs, and training runs. src/app.py is a database-backed game view that joins the latest scored model for a selected game and displays probability, score margin, and play log. These are internal engineering interfaces.

11.2 Public CourtVision interface

The redesigned public application uses a task-based sidebar rather than exposing all filters globally. Data and model resources are cached. Derived game summaries calculate final scores, probability swing, lead changes, largest leads, play counts, and team aggregates. The visual system uses a dark navigation rail, orange analytical accent, card-based hierarchy, responsive layout, and explanatory microcopy.

Page	Primary question	Key interactions
Overview	What happened across the season?	Headline metrics, latest final, recent games, top teams
Game Explorer	How did one game evolve?	Team/month/swing filters, game choice, probability and margin timelines, play search
Teams	How does a team compare?	Team profile, record, scoring averages, recent results, league table
Prediction Lab	What does the model say now?	Named teams, period, clock, score, scoring-play flag
Model	Why should I trust or question it?	Metrics, training scale, feature set, limitations

11.3 Interaction design principles

- Progressive disclosure: overview first, details and raw features later.
- Contextual controls: game filters appear only in Game Explorer.
- Plain language: probability explanations accompany technical metrics.
- Immediate feedback: Prediction Lab reruns on every input change.
- Traceability: play log and model-input expander expose supporting detail.
- Risk framing: the interface explicitly rejects betting-advice interpretation.

11.4 Public runtime computation

The public app reads bundled games.csv and plays_with_predictions.csv, recomputes game/team aggregates in memory, and loads the XGBoost artifact for Prediction Lab inference. It does not call the FastAPI service or PostgreSQL. This is intentional deployment simplification, but means the public UI and API are parallel inference implementations. Shared packaging would reduce drift.

12. Containers and Runtime Topology

12.1 Full local Docker Compose

Service	Port	Responsibility	Dependency
db	5432	PostgreSQL 15 warehouse	None
api	8000	FastAPI model inference	Healthy db, though prediction itself is artifact-based
dashboard	8501	Database-backed Streamlit game UI	Healthy db
mlflow	5001 -> 5000	MLflow server process	None configured

The main Python image is based on python:3.11-slim, installs libgomp1 for XGBoost, installs requirements, copies source/model/artifacts, exposes 8000, and starts Uvicorn. Compose overrides that command for the Streamlit dashboard.

12.2 Public image

The public image copies only app.py, exported data, model, and artifacts. It exposes port 7860 and launches Streamlit. The requirements use xgboost-cpu rather than the GPU-capable Linux package, avoiding a very large NCCL dependency and reducing build size and time on ARM.

12.3 Health and restart semantics

docker-compose.public.yml maps host port 80 to Streamlit 7860, applies restart: unless-stopped, and checks /_stcore/health every 30 seconds after a 45-second start period. Docker health captures application readiness, while the deployment script independently polls the same endpoint from outside the instance.

12.4 Environment and secrets

Development defaults embed admin/password123 in connection strings and Compose environment values. These are acceptable only for isolated local learning. Production credentials should be injected through AWS Systems Manager Parameter Store or Secrets Manager, database ports should not be public, and images should run as a non-root user. The public snapshot avoids database secrets entirely.

13. AWS Deployment and Operations

13.1 Provisioning sequence

Step	Action	Reason
1	Authenticate selected AWS CLI profile	Bind actions to dedicated IAM identity
2	Resolve default VPC and subnet	Choose network placement
3	Create/reuse security group	Idempotent network policy
4	Open TCP 80 globally; TCP 22 to current IP	Public app, restricted administration
5	Create/reuse EC2 key pair	SSH and rsync access
6	Resolve Canonical Ubuntu 22.04 ARM AMI from SSM	Current architecture-compatible image
7	Create/reuse tagged t4g.small instance	Avoid duplicate instances on redeploy
8	Install Docker using user data	Bootstrap host runtime
9	Rsync public bundle and Compose file	Transfer deployment unit
10	Build/start container and poll health	Verify actual readiness

13.2 IAM boundary

A dedicated nba-courtvision-deployer user uses a customer-managed policy that allows EC2 discovery, instance/security-group/key-pair lifecycle, tagging, and reading the Canonical AMI parameter. This is better than reusing an unrelated QueueFlow identity. The policy still uses Resource: * for several EC2 mutations; production hardening should constrain regions, tags, instance types, and resource ARNs where AWS supports it.

13.3 Current deployment snapshot

As of 27 August 2026, the application was deployed in us-east-1 to instance i-0762b9ee04298eaab and externally verified at http://44.193.213.198. The address is a public instance IP, not an Elastic IP, so it can change after a stop/start cycle. The endpoint is HTTP only.

13.4 Production hardening path

- Allocate a stable address or use an Application Load Balancer.
- Attach a domain through Route 53 and an ACM certificate for HTTPS.
- Remove direct SSH by using AWS Systems Manager Session Manager.
- Bake or pull a versioned image from ECR rather than building on the instance.
- Send container/system logs and alarms to CloudWatch.
- Add budget alerts, uptime checks, patching, backups, and documented rollback.
- Prefer an Auto Scaling Group or managed container platform if availability matters.

13.5 Cost model

The running EC2 instance and its EBS storage incur charges. Data transfer may also incur charges. The script reuses the tagged instance but does not automatically stop it. A portfolio owner should set an AWS Budget and decide whether the site must remain continuously available.

14. Reliability, Security, and Failure Analysis

14.1 Failure-mode table

Failure	Observable symptom	Current response	Recommended improvement
ESPN timeout	Prefect task error	3 retries, 5-second delay	Backoff + jitter; source monitoring
Malformed event	Normalization/load exception	Run marked failed	Quarantine bad event; continue others
DB unavailable	Connection exception	Flow/API UI fails	Connection retry; alerts; managed DB
Missing model	API health says model_not_loaded	Prediction returns 503	Deployment gate on artifact hash
Schema drift	Missing columns or SQL error	Legacy play table rename only	Versioned migrations and contracts
Bad input semantics	Plausible but invalid probability	Range validation only	Cross-field validation
EC2 reboot	Temporary outage	Container restarts	Load balancer + multi-instance
Public IP changes	Old URL breaks	No mitigation	Elastic IP or DNS
Host compromise	Application/data exposure	SSH limited by current IP	SSM, least privilege, patching, non-root

14.2 Existing strengths

- Network retries and timeouts prevent indefinite fetch hangs.
- Transactional upserts make repeated loads safe.
- Ingest and training run tables preserve operational history.
- Health endpoints distinguish process presence from model readiness.
- Docker restart policy handles ordinary process exits.
- Dedicated deployment identity separates AWS concerns.
- Public serving avoids database credentials and open database ports.

14.3 Test posture

No automated test suite is present in the repository snapshot. The frontend was exercised through Streamlit's application test harness across all five pages, and the production ARM container passed its health endpoint before and after deployment. That is valuable deployment verification, but not a substitute for unit, contract, data-quality, and model-regression tests.

14.4 Minimum future test pyramid

Layer	Examples
Unit	Clock parsing, feature equations, percentile calculation, label construction
Data contract	Required ESPN fields, score types, unique keys, monotonic sequence
Integration	ETL into temporary PostgreSQL; prediction upsert; API model load
Model regression	Metric thresholds, probability range, feature schema/hash
UI	Page smoke tests, filter-empty state, Prediction Lab boundary inputs
Deployment	Container health, external HTTP, rollback to previous image

15. Reproduction Guide

15.1 Local environment

```
cd nbaAnalytics
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
docker compose up -d db
```

15.2 Ingest a date range

```
python src/etl_pipeline.py --start-date 2025-10-01 --end-date 2025-10-07
# Or one or more events:
python src/etl_pipeline.py --event-id 401859967
```

15.3 Train and inspect artifacts

```
python src/train_model.py --min-games 20 --test-size 0.2 --random-state 42
cat artifacts/training_metrics.json
head artifacts/feature_importance.csv
head artifacts/calibration_bins.csv
```

15.4 Score warehouse plays

```
python src/score_predictions.py
python src/score_predictions.py --game-id 401859967 --batch-size 5000
```

15.5 Run inference surfaces

```
python src/predict_model.py --period 4 --seconds-remaining 300 --home-score 98 --away-score 95
uvicorn src.api:app --host 0.0.0.0 --port 8000
streamlit run src/app.py --server.port 8501
```

15.6 Run the public app locally

```
cd hf_space
streamlit run app.py --server.port 7860
# Health: http://127.0.0.1:7860/_stcore/health
```

15.7 Deploy with the dedicated AWS profile

```
AWS_PROFILE=nba-courtvision AWS_REGION=us-east-1 ./deploy/deploy_public_ec2.sh
```

15.8 Verification checklist

- Confirm ingest_runs ends in success with expected event/play counts.
- Confirm training includes both labels in train and test games.
- Review metrics and calibration before replacing a model.
- Confirm model and feature column order match.
- Check API /health and Streamlit /_stcore/health.
- Open Overview, Game Explorer, Teams, Prediction Lab, and Model.
- Record the instance ID, current public endpoint, model hash, and export date.

16. Design Critique and Roadmap

16.1 Highest-priority correctness improvements

Priority	Change	Why it matters
P0	Correct overtime clock parsing and add boundary tests	Training feature semantics are currently wrong in OT
P0	Create chronological/rolling evaluation	Random game split does not prove future performance
P0	Version export, model, feature schema, and metrics together	Prevents silent artifact mismatch
P1	Add naive and heuristic baselines	Metrics need comparative context
P1	Centralize feature pipeline	Avoid drift across five inference/training paths
P1	Add migrations and persistent DB volume	Protect state and schema consistency
P1	Add HTTPS and stable DNS	Public trust and durable address
P2	Add SHAP, ablations, and segment calibration	Improve interpretability
P2	Add lineup, possession, rest, injury, and strength context	Increase predictive information
P2	Add CI/CD, ECR, CloudWatch, and rollback	Operational maturity

16.2 A stronger modeling experiment

Construct a chronological benchmark: train on earlier games, validate on a later contiguous window, and test on the newest season segment. Compare (a) unconditional home prior, (b) logistic regression using margin and time, (c) current XGBoost, and (d) XGBoost with team-strength and possession features. Report AUC, log loss, Brier, ECE, and calibration curves by quarter. Bootstrap games, not rows, for confidence intervals.

16.3 A stronger production architecture

A mature version would schedule ETL in Prefect, store data in managed PostgreSQL, version transformations and models in an artifact registry, publish a signed CPU image to ECR, deploy behind HTTPS, and use a stable API consumed by the frontend. A feature-schema version would accompany each request and prediction. Monitoring would cover source freshness, missing-field rate, inference latency, probability drift, and observed calibration.

16.4 What not to add first

Social-media scraping about players' personal lives would create noisy, ethically sensitive, difficult-to-validate features. Higher-value improvements are structured and measurable: injuries from reputable feeds, official lineups, rest days, opponent strength, possession, and temporal evaluation. Better evidence should precede broader data collection.

17. What This Project Teaches

17.1 System-design lessons

- A pipeline is a chain of contracts, not a collection of scripts.
- Durable raw data and idempotent writes make iteration safe.
- Operational metadata is part of the data model.
- Artifacts form an interface between training and serving.
- A cheap snapshot architecture can be correct for a portfolio workload.
- Cloud deployment includes identity, networking, bootstrap, health, and cost - not just code upload.

17.2 Machine-learning lessons

- Split on the unit of independence: games, not correlated play rows.
- Probabilities require proper scoring rules and calibration, not accuracy alone.
- Feature importance is not causality.
- Evaluation scope defines the claims a model may make.
- A single missing upstream field can determine inference coverage.
- Feature transformation must remain identical across training and every serving path.

17.3 Product lessons

- Navigation should follow user questions, not implementation modules.
- A probability becomes useful when paired with context and traceability.
- Model limitations belong in the product, not only in documentation.
- The public interface and internal operational interface serve different audiences.
- Good presentation cannot compensate for ambiguous data lineage; both matter.

17.4 The one-minute explanation

CourtVision fetches NBA schedules and play-by-play summaries from ESPN, normalizes them into a PostgreSQL warehouse, and records each ingestion run. It labels every eligible play by the game's final home-win outcome, engineers 12 score-and-time features, and trains XGBoost with entire games held out. It evaluates ranking and probability quality, saves model and analysis artifacts, and can score the warehouse or answer HTTP requests. For the public site, it exports games and scored plays into a self-contained Streamlit application, packages it in Docker, and deploys it to AWS EC2 with a dedicated IAM identity and external health verification.

Mastery checkpoint
If you can explain the paragraph above, draw Figure 1 from memory, and derive the twelve features and four evaluation metrics, you understand the project at system level.

Appendix A. Repository Map

Path	Role
src/etl_pipeline.py	Prefect extraction, normalization, upsert, run accounting
src/database.py	Async SQLAlchemy ORM declarations
src/features.py	Canonical single-state feature builder and ordered column list
src/train_model.py	Training frame, grouped split, XGBoost, metrics, artifacts
src/score_predictions.py	Vectorized warehouse scoring and version-aware upserts
src/predict_model.py	Local CLI inference
src/api.py	FastAPI request validation, health, synchronous prediction
src/benchmark_api.py	Sequential client-side latency/throughput benchmark
src/app.py	Database-backed game explorer
src/dashboard.py	Warehouse operations dashboard
Dockerfile	Full API image
docker-compose.yml	Local db/api/dashboard/MLflow topology
hf_space/app.py	Self-contained public CourtVision application
hf_space/data/*	Bundled public season and prediction snapshot
hf_space/models/*	Bundled XGBoost model
hf_space/artifacts/*	Bundled evaluation evidence
deploy/deploy_public_ec2.sh	Idempotent EC2 provision, sync, start, verification
deploy/docker-compose.public.yml	Public container port, restart, health
deploy/ec2_deployer_policy.json	Dedicated deployer's IAM permissions
docs/*	EC2 and Hugging Face runbooks

Appendix B. Glossary

Term	Meaning in this project
Artifact	Saved model or evaluation output used beyond the training process
Brier score	Mean squared error of binary outcome probabilities
Calibration	Agreement between predicted probability and observed frequency
Control plane	Mechanisms that schedule, secure, deploy, retry, and observe work
Data plane	Games, plays, features, labels, probabilities, and artifacts
ETL	Extract source JSON, transform to schema, load durable tables
Game-level split	Assign all plays of one game to only train or test
Idempotent	Safe to repeat without multiplying logical records
Inference	Apply a trained model to features to produce a probability
Log loss	Proper scoring rule that heavily penalizes confident errors
Model drift	Performance change as real data differs from training data
ROC AUC	Ranking discrimination across positive and negative examples
Upsert	Insert a new key or update the existing row on conflict
Win probability swing	Within-game maximum probability minus minimum probability

Study sequence

Pass	Focus	Checkpoint
1. Architecture	Figures 1 and 2, Sections 1-4	Draw the two planes and five tables
2. Data	Sections 3-6	Explain one ESPN play from JSON to feature row
3. ML	Sections 7-9	Explain split, boosting, all four metrics, importance caveat
4. Serving	Sections 10-13	Trace one Prediction Lab request and one AWS deployment
5. Critique	Sections 14-16	Name top three correctness and top three ops improvements
6. Rebuild	Section 15 + Appendix A	Run pipeline commands and locate every implementation

References

[1] Chanakya G. NBA Analytics Pipeline repository and source snapshot, inspected 27 August 2026. Project code, README, model artifacts, deployment scripts, and bundled datasets.

[2] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016. https://doi.org/10.1145/2939672.2939785

[3] XGBoost Developers. XGBoost Python API and parameter documentation. https://xgboost.readthedocs.io/

[4] scikit-learn Developers. Model evaluation: probabilistic metrics, ROC AUC, log loss, and Brier score. https://scikit-learn.org/stable/modules/model_evaluation.html

[5] PostgreSQL Global Development Group. PostgreSQL documentation: constraints, indexes, transactions, and JSON types. https://www.postgresql.org/docs/

[6] Prefect Technologies. Prefect flows, tasks, retries, and orchestration documentation. https://docs.prefect.io/

[7] FastAPI. Request validation, OpenAPI, application events, and deployment documentation. https://fastapi.tiangolo.com/

[8] Streamlit. Application, caching, charting, and deployment documentation. https://docs.streamlit.io/

[9] Docker. Dockerfile, Compose, restart policy, and container health documentation. https://docs.docker.com/

[10] Amazon Web Services. Amazon EC2 User Guide, IAM User Guide, Systems Manager Parameter Store, and VPC security-group documentation. https://docs.aws.amazon.com/

[11] ESPN site API endpoints consumed by the implementation: /apis/site/v2/sports/basketball/nba/scoreboard and /summary. These endpoints are treated as external source interfaces by the code; their stability and licensing should be reviewed before production use.

Document basis: repository state and deployed system observed on 27 August 2026. Statistics in Sections 5, 8, and 9 were recomputed or read from the bundled CSV/JSON/CSV artifacts. This paper documents the system as implemented, and explicitly labels recommended future changes.